Activity 4 Part 1 - useState()

Preparation:

Generate a react project using, make sure to change your folder name

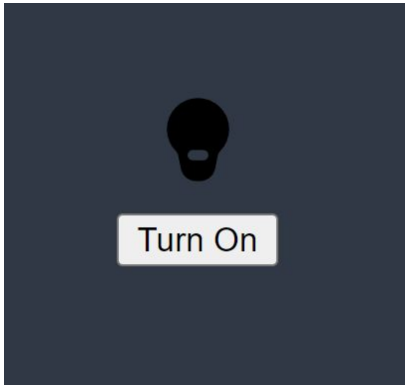
```
npm create vite@lastest react-hooks
```

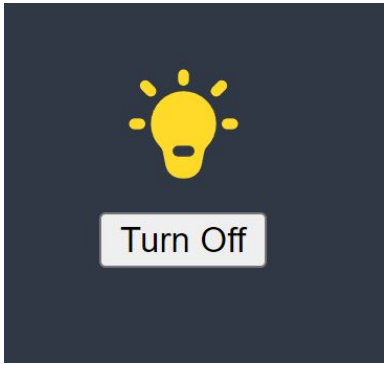
Instruction

- Create a new component, you can simply name them LightSwitch. Follow the convention
- Use useState hook to create a state variable, isLightOn and set an initial value to false.
- Render a button that will act as the light switch
- Display an image or text indicating whether the light is on or off, based on the state
- Write a function that toggles isLightOn from true to false and vice versa
    a. This function should be called when the button is clicked

Challenge yourself

- Change the background color of the page or a specific div based on whether the light is on or off





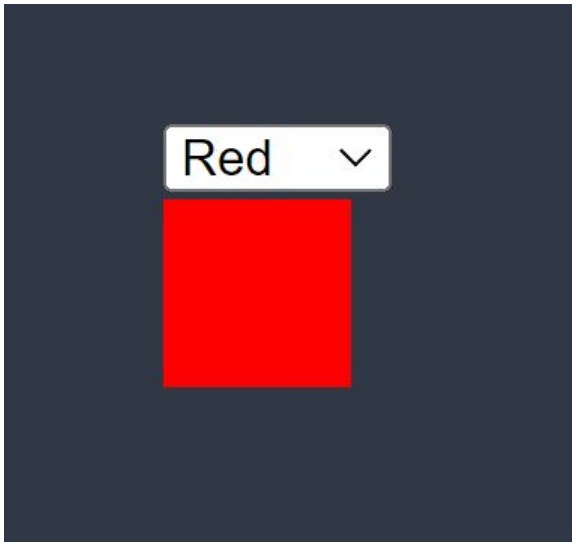
Activity 4 Part 2 - useState()

Instruction

- Create a component named ColorPicker. Follow convention
- Use and import the useState hook to create a state variable, initially set to a default color (e.g., 'red')
- Render a dropdown <select> element with a few color options (e.g., red, green, blue, etc.)
- Include a display box that shows the selected color
- Write a function to update the state variable, based on the user's selection
- Attach this function to the onChange event of the dropdown <select>

Challenge yourself

- Add more styling



Activity 4 Part 3 - useEffect()

Instruction

- Create a component called MousePosition. Follow convention.
- Define a state variable, position, with an initial value of an object x and y properties, each properties have a value of zero (0)
- Use the useEffect hook to update the state variable, position, whenever the mouse moves.
- Render the state variable

Example Implementation

```
import MousePosition from './components/MousePosition';

function App() {

  return (

    <MousePosition />

  );

}

export default App;
```

Activity 4 Part 4 - useEffect()

Instruction

- Create a component called RandomColor. Follow convention.
- Define a state variable, color, with an initial value of red or your favorite color.
- Use the useEffect hook to update the state variable, color, for every 3 seconds
- Render an element with an inline background color style that matches the current value of the state variable, color.

Example Implementation

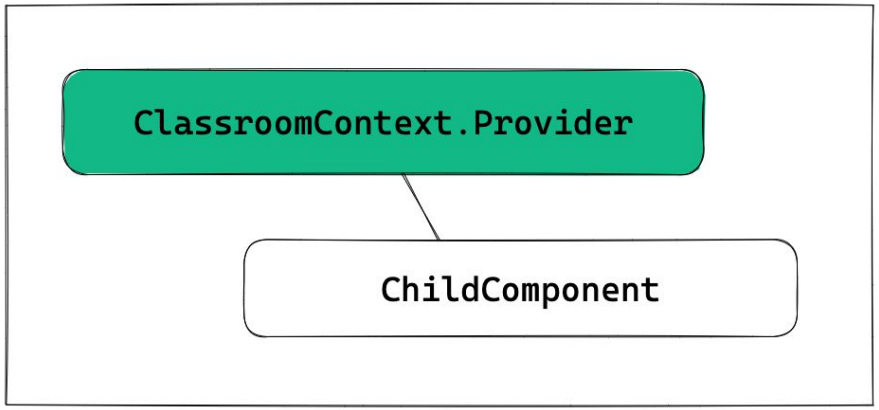
```
import RandomColor from
'./components/RandomColor';


function App() {
  return (
    <RandomColor />
  );
}
export default App;
```

Activity 4 Part 5 - useContext()

Scenario

Imagine you're building a virtual classroom app. Each student has their own username, and the app has a theme setting the determines the background color. You want to make sure that the username and theme are accessible to different components without passing them around manually



Instruction

- Create a context object using createContext(). Think of this as a special container where we'll store and share the username and theme data.
- Define the shared data object, which includes the username and theme in an object format
- Wrap your main component or the components that need access to the shared data with the Context.Provider component. Pass the shared data object as the value prop
- Create a component called ChildComponent
- Use the useContext() hook to access the shared data
- Display the username and theme data in the child component

Activity 4 Part 6 - useReducer()

Instruction

- Create a component called Form
- Define the initial state. The initial state will be an object with name and email properties
- Define a reducer function. The reducer takes the current state and an action as parameters, and returns the new state based on the action. We'll have the following actions:
  a. UPDATE_EMAIL
  b. UPDATE_NAME
  c. RESET_FORM
- **Remember NOT to modify the state**
- Use the useReducer hook. Pass the reducer function and the initial state as arguments to the hook. This hook will return an array with two (2) elements: the current state, and a dispatch function
- Render the Form